

数据结构与算法

第2章 线性表

张丽淼

光电信息获取与防护技术全国重点实验室

zhanglm@ahu.edu.cn

2026年3月25日

有序表



所谓**有序表**，是指这样的线性表，其中所有元素以**递增或递减**方式有序排列。

后面讨论的有序表默认元素是**以递增方式排列**。

例如： $L = (1, 5, 8, 10, 15, 20)$ 就是一个整数有序表。

有序表是线性表的一个子集，它继承了线性表的所有特性，但增加了**元素有序**这一约束。

有序表 $\subset$ 线性表

有序表



所谓**有序表**，是指这样的线性表，其中所有元素以**递增或递减**方式有序排列。

后面讨论的有序表默认元素是**以递增方式排列**。

例如： $L = (1, 5, 8, 10, 15, 20)$ 就是一个整数有序表。

注意

有序表和顺序表的区别

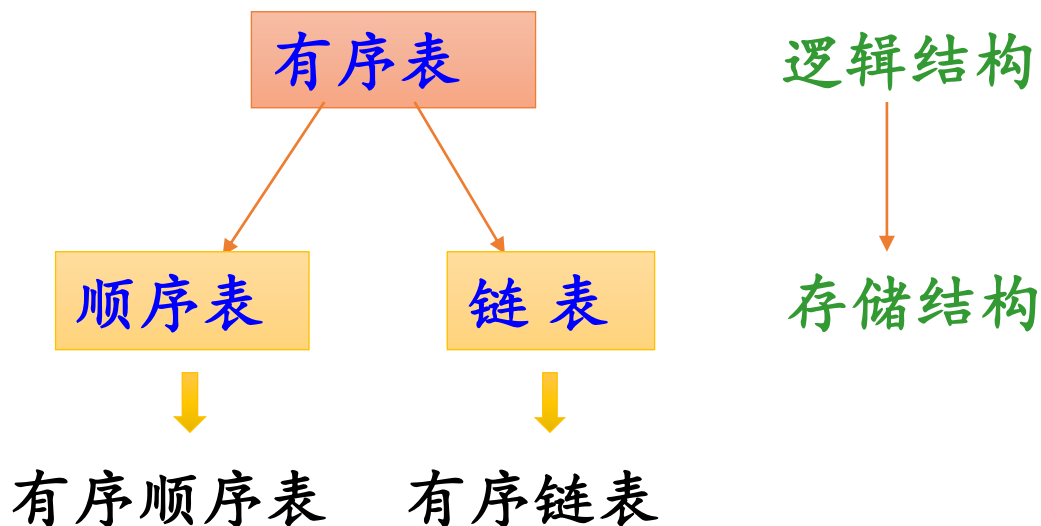
逻辑层面的概念

物理层面的概念

元素之间的逻辑关系相同，其区别是运算实现的不同。

有序表的存储结构及其基本运算

既然有序表中元素逻辑关系与线性表的相同，有序表可以采用与线性表相同的存储结构。



特别注意：**有序表和顺序表是两个不同的概念**。顺序表指的是物理存储结构（用数组存储），而有顺序表指的是逻辑上的有序性。一个表可以是有序的顺序表，也可以是有序的链表。

有序表的存储结构及其基本运算

- 在以顺序表或者链表存储有序表时，许多基本运算算法与线性顺序表或者链表的算法相同（不考虑优化）。
- 只有**插入**运算—ListInsert()算法有所差异。在有序表中，插入元素后必须保持表的有序性。

ListInsert(L, *i*, *e*)



线性表

ListInsert(L, *e*)



有序表

需要先找到正确的
插入位置

有序表的存储结构及其基本运算

若以**顺序表**存储有序表，ListInsert()算法算法如下：

```
void ListInsert(SqList *&L, ElemType e)
{ int i=0, j;
  while (i<L->length && L->data[i]<e)
    i++;                                //第一个大于等于e的元素的位置i
  for (j=ListLength(L);j>i;j--)        //将data[i..n]后移一个位置
    L->data[j]=L->data[j-1];
  L->data[i]=e;
  L->length++;                          //有序顺序表长度增1
}
```

*这里使用指针标识顺序表L, p34

有序表的存储结构及其基本运算

若以单链表存储有序表，ListInsert()的算法如下：

```
void ListInsert(LinkNode *&L, ElemType e)
{ LinkNode *pre=L, *p;

  while (pre->next!=NULL && pre->next->data<e)
    pre=pre->next; //查找插入结点的前驱结点pre

  p=(LinkNode *)malloc(sizeof(LinkNode));
  p->data=e;
  p->next=pre->next;
  pre->next=p;

}
```

在pre之后插入p结点

查找插入的位置pre

有序表的合并



【例2.14】假设有两个有序表LA和LB。设计一个算法，将它们合并成一个有序表LC。

核心思想：

同时遍历两个有序表LA和LB，比较当前元素的大小，将较小的那个元素放入新表LC中。当其中一个表遍历完毕后，将另一个表剩余的元素全部加入LC。



二路归并示意图

有序表的合并



重点
掌握

【例2.14】假设有两个有序表LA和LB。设计一个算法，将它们合并成一个有序表LC。

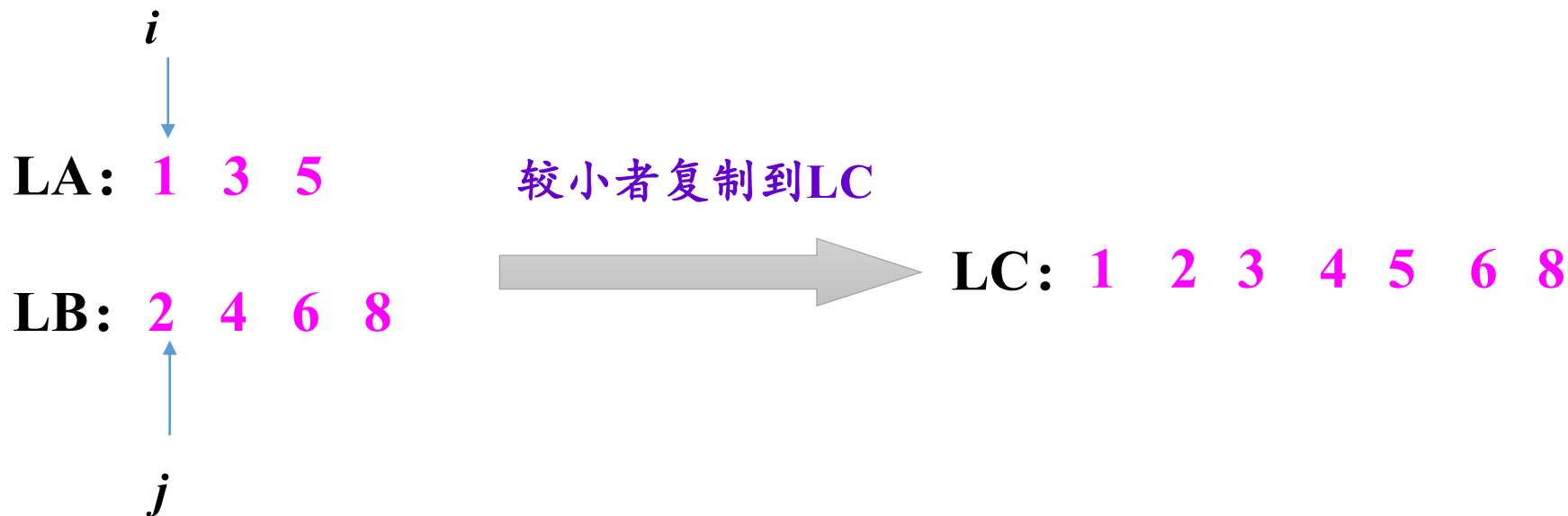
需要掌握：

- ✓ 能够手动画出二路归并的过程。
- ✓ 能够理解并写出顺序表和链表的归并算法。
- ✓ 掌握最好和最坏情况下的比较次数。

顺序表二路归并示例的演示

例如, $LA = (1, 3, 5)$, $LB = (2, 4, 6, 8)$

其二路归并过程如下:



LA 、 LB 中每个元素恰好遍历一次, 时间复杂度为 $O(m+n)$ 。

采用顺序表存放有序表时，二路归并算法如下：

```
void UnionList(SqList *LA, SqList *LB, SqList *&LC)
{ int i=0, j=0, k=0;    //i、j分别为LA、LB的下标，k为LC中元素个数
  LC=(SqList *)malloc(sizeof(SqList));    //建立有序顺序表LC
  while (i<LA->length && j<LB->length)
  { if (LA->data[i]<LB->data[j])
    { LC->data[k]=LA->data[i];
      i++;k++;
    }
    else    //LA->data[i]>LB->data[j]
    { LC->data[k]=LB->data[j];
      j++;k++;
    }
  }
}
```



两个有序表
均没有遍历
完

```
while (i<LA->length) //LA尚未扫描完，将其余元素插入LC中
{
    LC->data[k]=LA->data[i];
    i++;k++;
}
while (j<LB->length) //LB尚未扫描完，将其余元素插入LC中
{
    LC->data[k]=LB->data[j];
    j++;k++;
}
LC->length=k;
}
```

本算法的时间复杂度为 $O(m+n)$ ，空间复杂度为 $O(m+n)$ 。

采用单链表存放有序表时，二路归并算法如下：

```
void UnionList1(LinkNode *LA, LinkNode *LB, LinkNode *&LC)
{ LinkNode *pa=LA->next, *pb=LB->next, *r, *s;
  LC=(LinkNode *)malloc(sizeof(LinkNode));    //创建LC的头结点
  r=LC;                                         //r始终指向LC的尾结点
  while (pa!=NULL && pb!=NULL)
  { if (pa->data<pb->data)
    { s=(LinkNode *)malloc(sizeof(LinkNode)); //复制结点
      s->data=pa->data;
      r->next=s;r=s;                          //采用尾插法将s插入到LC中
      pa=pa->next;
    }
    else
    { s=(LinkNode *)malloc(sizeof(LinkNode)); //复制结点
      s->data=pb->data;
      r->next=s;r=s;                          //采用尾插法将s插入到LC中
      pb=pb->next;
    }
  }
}
```

```

while (pa!=NULL)
{ s=(LinkNode *)malloc(sizeof(LinkNode)); //复制结点
  s->data=pa->data;
  r->next=s;r=s; //采用尾插法将s插入到LC中
  pa=pa->next;
}

while (pb!=NULL)
{ s=(LinkNode *)malloc(sizeof(LinkNode)); //复制结点
  s->data=pb->data;
  r->next=s;r=s; //采用尾插法将s插入到LC中
  pb=pb->next;
}

r->next=NULL //尾结点的next域置空
}

```

本算法的时间复杂度为 $O(m+n)$ ，空间复杂度为 $O(m+n)$ 。

有序表的合并

- 上述两个算法的时间复杂度均为 $O(m+n)$ 。
- 主要的时间花费在元素比较上。
- 两个长度分别为 m 、 n 的有序表 A 和 B ，采用二路归并算法，**最好情况**下元素比较的次数（即最少的比较次数）是多少？

$\text{MIN}(m, n)$: 较短的表的所有元素都小于较长的表的第一个元素。

如 $A=(1, 2, 3)$, $B=(5, 6, 7, 8, 9)$, 元素比较次数为3。

- 两个长度分别为 m 、 n 的有序表 A 和 B ，采用二路归并算法，**最坏情况**下元素比较的次数（即最多的比较次数）是多少？

$m+n-1$: 如 $A=(2, 4, 6)$, $B=(1, 3, 5, 7)$, 元素比较次数为6。

思考1：要求合并后的表无重复数据，如何实现？

提示：要单独考虑

`pa->data == pb->data`

- ✓ 在归并过程中，如果遇到两个元素相等的情况，我们只取其中一个即可。具体实现时，在比较大小的代码中加入相等判断，复制其中一个后，两个指针都向后移动一位。

思考2：将两个非递减的有序链表合并为一个非递增的有序链表，如何实现？

- 要求结果链表仍使用原来两个链表的存储空间，不另外占用其它的存储空间。
- 表中允许有重复的数据。

提示：头插法

值大的放前面，值小的放后面

【算法步骤】

- (1) Lc指向La （空间复杂度均为 $O(1)$ ）
- (2) 依次从 La 或 Lb 中“摘取”元素值较小的结点插入到 Lc 表的表头结点之后，直至其中一个表变空为止
- (3) 继续将 La 或 Lb 其中一个表的剩余结点插入在 Lc 表的表头结点之后
- (4) 释放 Lb 表的表头结点

有序表的应用

【例2.16】已知一个有序单链表L（允许出现值域重复的结点），设计一个高效算法删除值域重复的结点。并分析算法的时间复杂度。

- 由于是有序单链表，所以相同值域的结点都是相邻的。
- 用 p 扫描递增单链表，若 p 所指结点的值域等于其后继结点的值域，则删除后者。

```

void dels(LinkNode *&L)
{
    LinkNode *p=L->next,*q;
    while (p->next!=NULL)
    { if (p->data==p->next->data) //找到重复值的结点
        { q=p->next;           //q指向这个重复值的结点
          p->next=q->next;      //删除q结点
          free(q);
        }
        else //不是重复结点,p指针下移
            p=p->next;
    }
}

```

算法的时间复杂度为 $O(n)$

【例2.17】 一个长度为 L ($L \geq 1$) 的升序序列 S ，处在第 $\lceil L/2 \rceil$ 个位置的数称为 S 的中位数。

例如：若序列 $S_1=(11, 13, 15, 17, 19)$ ，则 S_1 的中位数是15。

两个序列的中位数是含它们所有元素的升序序列的中位数。例如，若 $S_2=(2, 4, 6, 8, 20)$ ，则 S_1 和 S_2 的中位数是11。

现有两个等长的升序序列 A 和 B ，试设计一个在时间和空间两方面都尽可能高效的算法，找出两个序列 A 和 B 的中位数。要求：

(1) 给出算法的基本设计思想。

(2) 根据设计思想，采用C、C++或Java语言描述算法，关键之处给出注释。

(3) 说明你所设计算法的时间复杂度和空间复杂度。

注：本题为2011年全国考研题。

解

$n=5$

$S_1=(11, 13, 15, 17, 19)$

$S_2=(2, 4, 6, 8, 20)$



$S=(2, 4, 6, 8, 11, 13, 15, 17, 19, 20)$

中位数

实际上，不要求出 S 的全部元素，用 k 记录当前归并的元素个数，当 $k==n$ 时，归并的那个元素就是中位数。

$n=5$

i


$S_1=(11, 13, 15, 17, 19)$

$k = 5$

$S_2=(2, 4, 6, 8, 20)$


 j



$k=n$, 结果为 i 、 j 所指较小的元素

中位数为 $S_1[i]=11$

对应的算法如下：

```
ElemType M_Search(SqList *A, SqList *B) //A、B的长度相同
{ int i=0, j=0, k=0;
  while (i<A->length && j<B->length) //两个序列均没有扫描完
  { k++;                               //当前归并的元素个数增1
    if (A->data[i]<B->data[j])         //归并较小的元素A->data[i]
    { if (k==A->length)                 //若当前归并的是第n个元素
      return A->data[i];               //返回A->data[i]
      i++;
    }
    else                               //归并较小的元素B->data[j]
    { if (k==B->length)                 //若当前归并的是第n个元素
      return B->data[j];               //返回B->data[j]
      j++;
    }
  }
}
```

算法的时间复杂度为 $O(n)$ ，空间复杂度为 $O(1)$ 。

线性表两类存储结构的比较

存储结构 比较项目		顺序表	链表
空间	存储空间	预先分配，会导致空间闲置或溢出现象	动态分配，不会出现存储空间闲置或溢出现象
	存储密度	不用为表示结点间的逻辑关系而增加额外的存储开销，存储密度等于1	需要借助指针来体现元素间的逻辑关系，存储密度小于1
时间	存取元素	随机存取，按位置访问元素的时间复杂度为 $O(1)$	顺序存取，按位置访问元素时间复杂度为 $O(n)$
	插入、删除	平均移动约表中一半元素，时间复杂度为 $O(n)$	不需移动元素，确定插入、删除位置后，时间复杂度为 $O(1)$
适用情况		① 表长变化不大，且能事先确定变化的范围 ② 很少进行插入或删除操作，经常按元素位置序号访问数据元素	① 长度变化较大 ② 频繁进行插入或删除操作

作业：

书 P72, 12、 17、 19

ddl: 4.1